

28 — JWT Authentication and User Management

Requirement: R26 (Admin UI authentication) **Phase:** L

What it is

When you build a web application with an admin panel, you need a way to verify that the person using it is authorized. You cannot just leave an admin dashboard open to the internet. This project uses **JWT (JSON Web Token) authentication** combined with **bcrypt password hashing** to protect the Admin UI.

Here are the core concepts:

Password hashing with bcrypt

When a user creates an account, you never store their password in plain text. Instead, you run it through a **one-way hash function** — a mathematical process that converts the password into a fixed-length string of random-looking characters. "One-way" means you cannot reverse the process to get the original password back.

bcrypt is the industry standard for password hashing. It has two critical features that simpler algorithms like SHA-256 lack:

1. **Built-in salt.** A salt is a random string mixed into the password before hashing. Without a salt, two users with the same password would produce the same hash — an attacker who cracks one cracks both. bcrypt generates a unique salt for every password automatically.
2. **Adjustable work factor.** bcrypt is intentionally slow. The `gensalt()` function accepts a "rounds" parameter (default: 12) that controls how many iterations the algorithm runs. Each increment doubles the computation time. SHA-256 is designed to be fast (good for file checksums), but speed is the enemy of password security — a fast algorithm lets an attacker try billions of guesses per second. bcrypt at 12 rounds takes roughly 250 milliseconds per hash, making brute-force attacks impractical.

JWT (JSON Web Token)

Once a user logs in with the correct password, the server needs a way to remember "this person is authenticated" for subsequent requests. Traditional web apps use server-side sessions (stored in memory or a database), but that requires the server to look up session data on every request.

A JWT takes a different approach: the server creates a **self-contained token** that carries all the information needed to verify the user. The token has three parts:

Part	Contents	Purpose
Header	<code>{"alg": "HS256", "typ": "JWT"}</code>	Tells the decoder which signing algorithm was used
Payload	<code>{"sub": "admin", "role": "admin", "exp": 1720000000}</code>	The actual claims — who the user is, what role they have, when the token expires
Signature	HMAC-SHA256 of header + payload	Proves the token was issued by this server and has not been tampered with

The three parts are base64-encoded and joined with dots:

`header.payload.signature`. The client stores this token (typically in `localStorage`) and sends it with every request in the `Authorization: Bearer <token>` header.

The key insight: the server never needs to store session state. It can verify any token by checking the signature against its secret key. If the signature is valid and the token has not expired, the user is authenticated.

Seed admin pattern

A chicken-and-egg problem: if the Admin UI requires login, and users are created through the Admin UI, how do you create the first user? The **seed admin pattern** solves this by creating a default admin user from environment variables when the application starts. The password hash is pre-generated and stored in `.env`, not the raw password.

CRUD user management

Once you have authentication, you can build endpoints for managing users — listing all users, creating new ones, and deleting existing ones. These endpoints are themselves protected by JWT auth, so only authenticated admins can manage other users.

Real-life analogy

Think of a building with a secured office floor.

Password hashing is like the building's key card system. When you register as an employee, the security office does not write down your face and keep a photo on file — that would be like storing a plain text password. Instead, they encode your biometric data into a key card. Even if someone steals the filing cabinet of key card records, they cannot reconstruct your face from the encoded data.

bcrypt vs SHA-256 is the difference between a combination lock with 4 digits vs 12 digits. SHA-256 is like the 4-digit lock — technically secure, but a determined person with a computer can try all 10,000 combinations in seconds. bcrypt is like a 12-digit lock that also forces you to wait 1 second between each attempt. Suddenly trying every combination takes years instead of seconds.

JWT is like a visitor badge that says "John Smith, Floor 3 access, valid until 5pm, signed by Front Desk." Every guard on every floor can read the badge and verify the front desk's signature stamp without calling the front desk to check. The badge is self-contained — all the information is right there. If someone alters "Floor 3" to "Floor 7", the signature will not match, and the guard will reject it.

Seed admin is like the building manager who has a master key from day one, before any employees are registered. Without this, nobody could get into the security office to register the first employee.

Why we used it here

The Bilingual Search Admin UI (R26) needs authentication so only authorized users can change AI provider settings, edit system prompts, and manage the ETL pipeline. The search API endpoints themselves do not require auth (SPEC.md section 3 explicitly leaves this as a gap for v1), but the admin panel does.

The architecture decisions:

1. **bcrypt over SHA-256** — SHA-256 is a general-purpose hash designed for speed. That is exactly what you do not want for passwords. bcrypt's built-in salt and adjustable cost factor make it the correct choice for credential storage.
 2. **JWT over server sessions** — the Admin UI is a React SPA that makes stateless API calls. JWT fits naturally because the React app stores the token in `localStorage` and attaches it to every request. No server-side session store needed.
 3. **Seed admin from environment** — the first deployment needs to work immediately. The `ADMIN_USERNAME` and `ADMIN_PASSWORD_HASH` environment variables let you set up the initial admin without manual database access.
 4. **CRUD endpoints behind `require_admin`** — user management (list, create, delete) uses FastAPI's dependency injection system. The `require_admin` function is injected into every protected route, decoding and verifying the JWT before the route handler runs.
-

Code walkthrough

1. Password verification with bcrypt (`search_api/routers/auth.py`)

When a user submits their username and password, the system checks both the database and the environment-variable fallback:

```
def _authenticate_user(username: str, password: str) -> str | None:
    try:
        conn = get_connection()
        try:
            _ensure_users_table(conn)
            with conn.cursor() as cur:
                # Look up the user's stored hash from the database
                cur.execute(
                    "SELECT password_hash, role FROM users WHERE username = %s",
                    (username,)
                )
                row = cur.fetchone()
                # bcrypt.checkpw: encode the plain password to bytes,
                # then compare it against the stored hash.
                # bcrypt extracts the salt from the stored hash automatically –
                # it knows the first 29 characters are the salt prefix.
```

```

        if row and bcrypt.checkpw(password.encode(), row[0].encode()):
            return row[1] # Return the user's role ("admin")
    finally:
        conn.close()
except Exception:
    pass

# Fallback: check the env-var seed admin credentials
# This ensures login works even if the users table is empty
if username == ADMIN_USERNAME and ADMIN_PASSWORD_HASH:
    if bcrypt.checkpw(password.encode(), ADMIN_PASSWORD_HASH.encode()):
        return "admin"
return None # Authentication failed

```

The critical detail: `bcrypt.checkpw` does not decrypt the hash (that is impossible with a one-way function). Instead, it hashes the provided password with the same salt embedded in the stored hash and checks if the results match. If they do, the password is correct.

2. JWT token creation (`search_api/routers/auth.py`)

After successful authentication, the server creates a JWT:

```

@router.post("/login", response_model=LoginResponse)
def login(req: LoginRequest) -> LoginResponse:
    role = _authenticate_user(req.username, req.password)
    if not role:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid credentials",
        )

    now = datetime.now(timezone.utc)
    payload = {
        "sub": req.username, # "sub" = subject (who the token is about)
        "role": role,        # the user's role for authorization checks
        "iat": now,          # "iat" = issued at (when the token was created)
        "exp": now + timedelta(hours=JWT_EXPIRY_HOURS), # auto-expires after 24h
    }
    # jwt.encode signs the payload with our secret key using HMAC-SHA256.
    # Anyone can decode the payload (it is just base64), but only our
    # server can create a valid signature because only we know JWT_SECRET.
    token = jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGORITHM)

    logger.info(f"EVIDENCE_AUTH_LOGIN: user={req.username}")

```

```

return LoginResponse(
    access_token=token,
    expires_in=JWT_EXPIRY_HOURS * 3600, # in seconds for the client
)

```

The `exp` claim is key: the token self-destructs after 24 hours. The server does not need to maintain a blacklist or expiry table — the expiration is baked into the token itself.

3. JWT verification as a FastAPI dependency (`search_api/routers/auth.py`)

Every protected endpoint uses `require_admin` as a dependency:

```

security = HTTPBearer() # Extracts the Bearer token from the Authorization header

def require_admin(
    credentials: HTTPAuthorizationCredentials = Depends(security),
) -> str:
    try:
        # jwt.decode verifies the signature AND checks expiration.
        # If the signature doesn't match JWT_SECRET, it raises InvalidTokenError.
        # If "exp" is in the past, it raises ExpiredSignatureError.
        payload = jwt.decode(
            credentials.credentials,
            JWT_SECRET,
            algorithms=[JWT_ALGORITHM],
        )
        return payload["sub"] # Return the username for logging
    except jwt.ExpiredSignatureError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token expired",
        )
    except jwt.InvalidTokenError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid token",
        )

```

FastAPI's `Depends()` system runs this function before the route handler. If `require_admin` raises an `HTTPException`, FastAPI short-circuits the request and returns 401. The route handler never executes.

4. Seed admin on startup (`search_api/routers/auth.py`)

```
def seed_admin_user():
    """Create the default admin user from env vars if the users table is empty."""
    if not ADMIN_PASSWORD_HASH:
        return # No seed credentials configured – skip silently
    try:
        conn = get_connection()
        try:
            _ensure_users_table(conn)
            with conn.cursor() as cur:
                cur.execute("SELECT COUNT(*) FROM users")
                if cur.fetchone()[0] == 0:
                    # Only seed if the table is completely empty.
                    # Once a real user exists, the seed is no longer needed.
                    cur.execute(
                        "INSERT INTO users (username, password_hash, role) "
                        "VALUES (%s, %s, %s)",
                        (ADMIN_USERNAME, ADMIN_PASSWORD_HASH, "admin"),
                    )
                    conn.commit()
                    logger.info(
                        f"EVIDENCE_AUTH_SEED: seeded default admin user "
                        f"'{ADMIN_USERNAME}'"
                    )
        finally:
            conn.close()
    except Exception as e:
        logger.warning(f"Could not seed admin user: {e}")
```

This function runs once at application startup. It checks if the users table is empty and, if so, inserts the admin user from environment variables. The `ADMIN_PASSWORD_HASH` is pre-generated with bcrypt (e.g., via `python -c "import bcrypt; print(bcrypt.hashpw(b'mypassword', bcrypt.gensalt()).decode())"`) — the raw password never appears in the codebase or environment.

5. User CRUD endpoints (`search_api/routers/auth.py`)

Creating a new user hashes the password before storage:

```
@router.post("/users", response_model=UserResponse, status_code=201)
def create_user(req: CreateUserRequest, admin: str = Depends(require_admin)):
    # bcrypt.hashpw: hash the password with a freshly generated salt.
    # gensalt() creates a random 16-byte salt with default 12 rounds.
    # The result includes the algorithm identifier, cost factor, salt,
```

```

# and hash – all in one string like "$2b$12$salt...hash..."
hashed = bcrypt.hashpw(req.password.encode(), bcrypt.gensalt()).decode()
conn = get_connection()
try:
    _ensure_users_table(conn)
    with conn.cursor() as cur:
        cur.execute(
            "SELECT 1 FROM users WHERE username = %s", (req.username,)
        )
        if cur.fetchone():
            raise HTTPException(
                status_code=409, detail="Username already exists"
            )
        cur.execute(
            "INSERT INTO users (username, password_hash, role) "
            "VALUES (%s, %s, %s) RETURNING id, created_at",
            (req.username, hashed, req.role),
        )
        row = cur.fetchone()
        conn.commit()
    logger.info(
        f"EVIDENCE_AUTH_CREATE_USER: admin={admin} "
        f"created user={req.username}"
    )
    return UserResponse(
        id=row[0],
        username=req.username,
        role=req.role,
        created_at=str(row[1]),
    )
finally:
    conn.close()

```

Notice the self-deletion guard in the delete endpoint:

```

@router.delete("/users/{username}", status_code=204)
def delete_user(username: str, admin: str = Depends(require_admin)):
    if username == admin:
        raise HTTPException(
            status_code=400, detail="Cannot delete yourself"
        )
    # ... delete logic

```

This prevents an admin from accidentally locking everyone out by deleting their own account.

How to explain this in an interview

"We use JWT authentication with bcrypt password hashing to protect the Admin UI. When a user logs in, bcrypt verifies their password by re-hashing it with the stored salt and comparing the results — the actual password is never stored or even reversible from the hash. On success, the server issues a JWT containing the username, role, and expiration time, signed with HMAC-SHA256. The React frontend stores this token in localStorage and attaches it to every API request as a Bearer token. On the server side, a FastAPI dependency called `require_admin` intercepts every protected endpoint, decodes the JWT, verifies its signature and expiration, and either returns the authenticated username or rejects the request with a 401. For first-time deployment, a seed admin pattern creates the initial user from environment variables so the system is immediately usable without manual database access."